

Initiation à l'environnement Unix

CM2 : Manipuler les fichiers dans l'arborescence

Pierre Rousselin
`rousselin@univ-paris13.fr`

Université Sorbonne Paris Nord
L1 informatique et double-licence
septembre 2024

Documentation des commandes dans le terminal

L'arborescence des fichiers

Manipuler les fichiers

Caractères joker et motifs du shell

Développement de noms de chemins

man man

Obtenir de l'aide :

- ▶ pour un programme externe : `man`
- ▶ pour une primitive du shell, dans `bash`, `help`
- ▶ pour de nombreuses commandes (non standard), option `--help`

```
$ type type
```

```
type est une primitive du shell
```

```
$ help type
```

```
type: type [-afptP] nom [nom ...]
```

```
Affiche des informations sur le type de commande.
```

```
Pour chaque NOM, indique comment il serait  
interprete s il etait utilise comme un nom de  
commande.
```

```
...
```

```
$ type id
```

```
id est /usr/bin/id
```

```
$ man id
```

```
$ type man
```

```
man est /usr/bin/man
```

```
$ man man
```

Section, nom et synopsis

- ▶ Le manuel est organisé en sections (voir `man man` et TP).
- ▶ La section est entre parenthèses après le nom de commande en haut à gauche de la page.
- ▶ La partie « NOM » décrit en une ligne ce que fait la commande.
- ▶ La partie « SYNOPSIS » décrit les syntaxes acceptées par la commande.

`entre crochets` optionnel

`avant ...` peut être répété

| indique un « ou exclusif »

- ▶ Les pages de manuel sont plus ou moins compréhensibles...
- ▶ ... mais on a rarement besoin de les comprendre en entier.

Aide, encore

- ▶ La plupart des commandes GNU ont une option `--help` qui donne une aide succincte.

```
$ who --help
```

```
Usage: who [OPTION]... [ FICHER | ARG1 ARG2 ]
```

```
Afficher des informations sur les utilisateurs  
connectés.
```

```
...
```

Aide, encore

- ▶ La plupart des commandes GNU ont une option `--help` qui donne une aide succincte.

```
$ who --help
```

```
Usage: who [OPTION]... [ FICHER | ARG1 ARG2 ]
```

```
Afficher des informations sur les utilisateurs  
connectés.
```

```
...
```

- ▶ `man -k mot` imprime les commandes dont la description succincte contient ce mot.

```
$ man -k chercher
```

```
apropos (1)           - Chercher le nom et la  
  description des pages de manuel
```

```
badblocks (8)         - Rechercher des blocs  
  défectueux sur un périphérique
```

```
...
```

Et sur le web ?

Pourquoi pas... toujours mieux que bloquer dans son coin ! Mais :

- ▶ il est tout de même indispensable de savoir obtenir de l'aide hors connexion ;
- ▶ les informations données sur votre machine sont certainement les plus adaptées aux logiciels installés sur votre machine ;
- ▶ **Ne jamais copier-coller aveuglément une commande !**
- ▶ qualité des informations non homogène sur le web (vous pouvez trouver le pire comme le meilleur)

Aide, encore

- ▶ Si elle est installée, la commande `whatis` permet d'obtenir une description brève d'une commande ou d'une fonction, ainsi que sa section de manuel.

```
$ whatis printf
```

```
printf (1)    - Formatter et afficher des donnees
```

```
printf (3)    - formatted output conversion
```


Documentation des commandes dans le terminal

L'arborescence des fichiers

Manipuler les fichiers

Caractères joker et motifs du shell

Développement de noms de chemins

Répertoires et fichiers normaux

Parmi les différents types de fichiers, on trouve :

les répertoires (*directories*) qui contiennent seulement des informations sur les fichiers qu'ils « contiennent » ;

les fichiers normaux (*regular files*) qui contiennent des données (fichier source C, image au format jpg, programme compilé, ...).

Répertoires et fichiers normaux

Parmi les différents types de fichiers, on trouve :

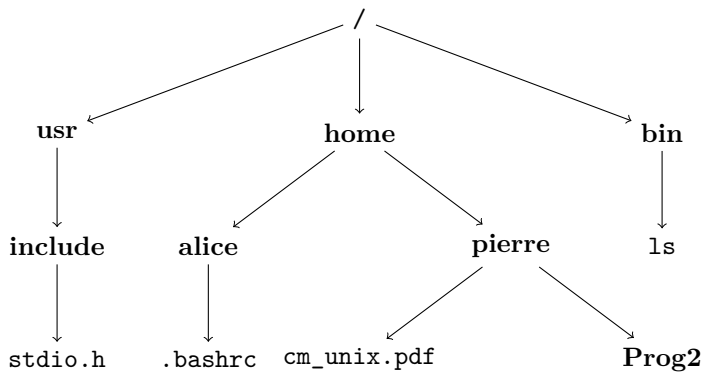
les **répertoires** (*directories*) qui contiennent seulement des informations sur les fichiers qu'ils « contiennent » ;

les **fichiers normaux** (*regular files*) qui contiennent des données (fichier source C, image au format jpg, programme compilé, ...).

Ces fichiers sont

- ▶ organisés sous la forme d'une unique arborescence ;
- ▶ dont les nœuds internes sont des répertoires ;
- ▶ dont les fichiers non répertoires sont les feuilles ;
- ▶ dont la racine de l'arborescence est le répertoire / ;
- ▶ dans laquelle chaque répertoire a un répertoire parent ;
- ▶ où la racine est le seul répertoire qui est son propre parent.

Un exemple



Chemin absolu

Un chemin absolu est

- ▶ une chaîne de caractères ;
- ▶ qui commence par / ;
- ▶ où les différents éléments du chemin sont séparés par / ;
- ▶ où chaque fichier du chemin est situé dans le répertoire qui le précède immédiatement dans ce chemin.

Exemples :

- ▶ `/home/pierre/cm_unix.pdf`
- ▶ `/usr`
- ▶ `/usr/`
- ▶ `/usr/include/`
- ▶ `/`

Si un **répertoire** se trouve à la fin du chemin, il peut être suivi ou non du caractère /

Chemin absolu

Un chemin absolu est un chemin qui commence par `/`.

Chemin absolu

Un chemin absolu est un chemin qui commence par `/`.

Un chemin qui commence par `/` est un chemin absolu.

Chemin absolu

Un chemin absolu est un chemin qui commence par /.

Un chemin qui commence par / est un chemin absolu.

Un chemin absolu est un chemin qui commence par /.

Chemin absolu

Un chemin absolu est un chemin qui commence par `/`.

Un chemin qui commence par `/` est un chemin absolu.

Un chemin absolu est un chemin qui commence par `/`.

Un chemin qui commence par `/` est un chemin absolu.

Chemin absolu

Un chemin absolu est un chemin qui commence par `/`.

Un chemin qui commence par `/` est un chemin absolu.

Un chemin absolu est un chemin qui commence par `/`.

Un chemin qui commence par `/` est un chemin absolu.

Un chemin absolu est un chemin qui commence par `/`.

Chemin absolu

Un chemin absolu est un chemin qui commence par `/`.

Un chemin qui commence par `/` est un chemin absolu.

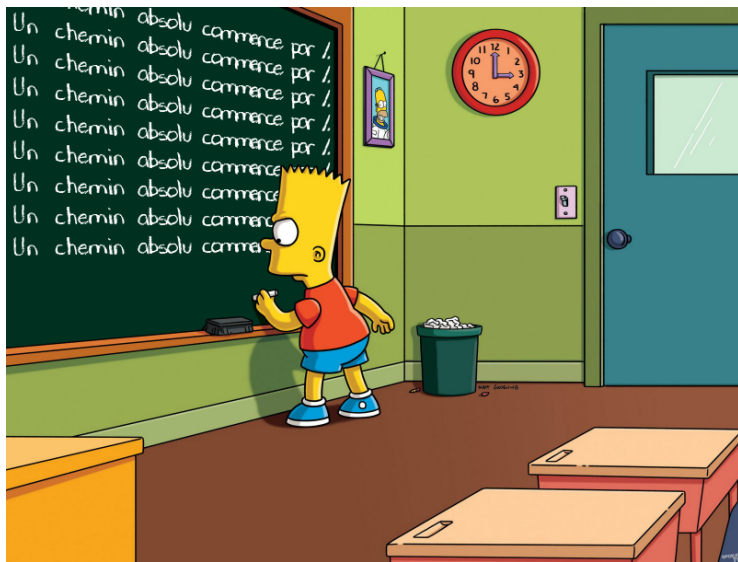
Un chemin absolu est un chemin qui commence par `/`.

Un chemin qui commence par `/` est un chemin absolu.

Un chemin absolu est un chemin qui commence par `/`.

Un chemin qui commence par `/` est un chemin absolu.

Chemin absolu



Généré par <https://www.ranzey.com/generators/bart/index.html>

. et ..

Dans un chemin, le point (.) désigne le répertoire qui le précède immédiatement, tandis que deux points successifs (..) désigne le parent du répertoire qui le précède. Exemples :

- ▶ `/home/pierre/../alice` `/home/alice`
- ▶ `/usr/include/../../` `/`
- ▶ `/usr/./include/./` `/usr/include`
- ▶ `/usr/../../home/../../bin/./` `/bin`

Répertoire courant et chemin relatif

À tout moment, le shell¹ enregistre le chemin absolu d'un répertoire qu'on appelle **répertoire courant** (*working directory*).

La commande **pwd** (pour *print working directory*) permet de l'afficher dans le terminal.

- ▶ Un chemin qui **ne commence pas par /** est un **chemin relatif** au répertoire courant.
- ▶ Le **chemin absolu correspondant** est obtenu en concaténant le **chemin absolu du répertoire courant** et le **chemin relatif**.

1. en fait, n'importe quel processus

Répertoire courant et chemin relatif

Exemples, si `/home/pierre` est le répertoire courant :

▶ <code>cm_unix.pdf</code>	<code>/home/pierre/cm_unix.pdf</code>
▶ <code>Prog2/</code>	<code>/home/pierre/Prog2/</code>
▶ <code>..</code>	<code>/home/</code>
▶ <code>../alice</code>	<code>/home/alice</code>
▶ <code>.</code>	<code>/home/pierre</code>
▶ <code>./cm_unix.pdf</code>	<code>/home/pierre/cm_unix.pdf</code>
▶ <code>../../usr/include/stdio.h</code>	<code>/usr/include/stdio.h</code>

Répertoire courant et chemin relatif

- ▶ Pour changer de répertoire courant :

```
cd chemin_rep
```

- ▶ exemples :

- ▶

```
cd Prog2
```

- ▶

```
cd ../../alice
```

- ▶

```
cd /usr/include
```

Partout où un chemin est attendu, on peut utiliser soit un chemin relatif, soit un chemin absolu !

Répertoire personnel

- ▶ Chaque utilisateur du système a un répertoire personnel (*home directory*).
- ▶ Au démarrage, le répertoire courant est le répertoire personnel.
- ▶ La commande `cd` sans argument fait du répertoire personnel le répertoire courant.
- ▶ S'il est en début de mot, le caractère `~` (tilde, **Alt Gr-2**) est remplacé **par le shell** par le chemin absolu du répertoire personnel de l'utilisateur : c'est le **développement du tilde**.

Répertoire personnel

- ▶ Chaque utilisateur du système a un répertoire personnel (*home directory*).
- ▶ Au démarrage, le répertoire courant est le répertoire personnel.
- ▶ La commande `cd` sans argument fait du répertoire personnel le répertoire courant.
- ▶ S'il est en début de mot, le caractère `~` (tilde, **Alt Gr-2**) est remplacé **par le shell** par le chemin absolu du répertoire personnel de l'utilisateur : c'est le **développement du tilde**.

Il arrive que le shell transforme la ligne de commande avant que la commande soit lancée.

Exemples avec développement du tilde

Dans chaque cas suivant, qu'affiche la dernière commande ?

1.

```
$ echo ~  
/home/alice  
$ cd  
$ cd ../bob/docs  
$ pwd
```

Exemples avec développement du tilde

Dans chaque cas suivant, qu'affiche la dernière commande ?

1. `$ echo ~`

`/home/alice`

`$ cd`

`$ cd ../bob/docs`

`$ pwd`

Réponse : `/home/bob/docs`

Exemples avec développement du tilde

Dans chaque cas suivant, qu'affiche la dernière commande ?

1. `$ echo ~`

`/home/alice`

`$ cd`

`$ cd ../bob/docs`

`$ pwd`

Réponse : `/home/bob/docs`

2. `$ echo ~`

`/users/licence/bob`

`$ cd ~/mes_scripts/`

`$ pwd`

Exemples avec développement du tilde

Dans chaque cas suivant, qu'affiche la dernière commande ?

1. `$ echo ~`

`/home/alice`

`$ cd`

`$ cd ../bob/docs`

`$ pwd`

Réponse : `/home/bob/docs`

2. `$ echo ~`

`/users/licence/bob`

`$ cd ~/mes_scripts/`

`$ pwd`

Réponse : `/users/licence/bob/mes_scripts`

Exemples avec développement du tilde

Dans chaque cas suivant, qu'affiche la dernière commande ?

1. `$ echo ~`

`/home/alice`

`$ cd`

`$ cd ../bob/docs`

`$ pwd`

Réponse : `/home/bob/docs`

2. `$ echo ~`

`/users/licence/bob`

`$ cd ~/mes_scripts/`

`$ pwd`

Réponse : `/users/licence/bob/mes_scripts`

3. `$ echo ~`

`/users/licence/bob`

`$ cd /usr`

`$ pwd`

Exemples avec développement du tilde

Dans chaque cas suivant, qu'affiche la dernière commande ?

1. `$ echo ~`

`/home/alice`

`$ cd`

`$ cd ../bob/docs`

`$ pwd`

Réponse : `/home/bob/docs`

2. `$ echo ~`

`/users/licence/bob`

`$ cd ~/mes_scripts/`

`$ pwd`

Réponse : `/users/licence/bob/mes_scripts`

3. `$ echo ~`

`/users/licence/bob`

`$ cd /usr`

`$ pwd`

Réponse : `/usr`

Lister le contenu d'un répertoire

La commande `ls` (pour *list*) permet de lister le contenu d'un ou plusieurs répertoire et/ou d'afficher des informations sur les fichiers.

- ▶ Sans argument, elle liste le contenu du répertoire courant.
- ▶ Pour chaque argument qui est un chemin de répertoire, elle liste le contenu de ce répertoire.
- ▶ Pour chaque argument qui est un chemin de fichier non répertoire, elle l'affiche.
- ▶ Des tonnes (trop!) d'options...
- ▶ À utiliser énormément en mode interactif, mais (presque) jamais dans les scripts à cause de problèmes de portabilité (par exemple le `ls` qu'on trouve sous GNU/Linux est différent de celui de MacOS).

Quelques options standard de `ls`

- ▶ `-l` : affichage *long*, chaque fichier sur une ligne, avec les permissions, le propriétaire, la taille, etc ;
- ▶ `-a` : pour *all*, afficher aussi les fichiers cachés, c'est-à-dire ceux dont le nom commence par un point (.) ;
- ▶ `-h` : pour *human-readable*, les tailles affichées avec l'option `l` sont données avec des unités (K, M, G, etc) pour être plus lisibles plutôt qu'en octets ;
- ▶ `-d` : pour *directory*, traiter les répertoires en argument comme des fichiers normaux, ne pas lister leur contenu ;
- ▶ `-t` : pour *time*, trier en fonction de la date de dernière modification ;
- ▶ `-r` : pour *reverse*, inverser le tri.

On peut **combiner** les options (c'est souvent le cas), par exemple :

```
ls -lhrt ~ /usr
```

Documentation des commandes dans le terminal

L'arborescence des fichiers

Manipuler les fichiers

Caractères joker et motifs du shell

Développement de noms de chemins

Créer et détruire des répertoires

- ▶ `mkdir rep...` (pour *make directory*)
crée les répertoires dont les chemins sont donnés en arguments
- ▶ échoue si le chemin désigne déjà un fichier existant
- ▶ `rmdir rep...` (pour *remove directory*)
détruit les répertoires **vides** dont les chemins sont donnés en arguments
- ▶ échoue si le chemin ne désigne pas un répertoire vide.

Ces commandes sont sûres.

Supprimer des fichiers

- ▶ `rm fichier...` (pour *remove*)
supprime les fichiers *normaux* dont les chemins sont donnés en argument
- ▶ échoue si un des chemins désigne un répertoire
- ▶ `rm -r fichier...` ou `rm -R fichier...` (pour *récuratif*)
supprime les fichiers ou répertoires (et tout ce qu'ils contiennent!) dont les chemins sont donnés en arguments.
- ▶ `rm` demande confirmation avant de supprimer un fichier protégé en écriture, sauf avec l'option `-f` (pour *force*).
- ▶ Les fichiers et répertoires supprimés le sont **définitivement**.

Cette commande est indispensable mais **très dangereuse**, surtout avec l'option `-r`.

Avec l'option `-i` (pour *interactif*), `rm` demande confirmation avant de supprimer chaque fichier.

Copier et déplacer des fichiers

- ▶ `cp src dest`

crée une copie du fichier dont le chemin est `src` vers le chemin `dest`

échoue si `src` désigne un répertoire, sauf si option `-R` (pour récursif).

- ▶ `cp sources... rep/`

Crée des copies des fichiers sources (avec le même nom de base) dans le répertoire `rep`, qui doit être un répertoire existant.

Copier et déplacer des fichiers

- ▶ **cp src dest**
crée une copie du fichier dont le chemin est **src** vers le chemin **dest**
échoue si **src** désigne un répertoire, sauf si option **-R** (pour récursif).
- ▶ **cp sources... rep/**
Crée des copies des fichiers **sources** (avec le même nom de base) dans le répertoire **rep**, qui doit être un répertoire existant.
- ▶ **mv src dest**
Déplace (*move*) le fichier dont le chemin est **src** vers le chemin **dest**
N'échoue pas si **src** est un répertoire.
C'est en fait un renommage.
- ▶ **mv sources... rep/**
Déplace les fichiers **sources** dans le répertoire **rep**, qui doit être un répertoire existant.

Copier et déplacer des fichiers

- ▶ `cp src dest`
crée une copie du fichier dont le chemin est `src` vers le chemin `dest`
échoue si `src` désigne un répertoire, sauf si option `-R` (pour récursif).
- ▶ `cp sources... rep/`
Crée des copies des fichiers `sources` (avec le même nom de base) dans le répertoire `rep`, qui doit être un répertoire existant.
- ▶ `mv src dest`
Déplace (*move*) le fichier dont le chemin est `src` vers le chemin `dest`
N'échoue pas si `src` est un répertoire.
C'est en fait un renommage.
- ▶ `mv sources... rep/`
Déplace les fichiers `sources` dans le répertoire `rep`, qui doit être un répertoire existant.
- ▶ **Commandes dangereuses** car peuvent écraser silencieusement un fichier déjà existant (mais option `-i`).

Documentation des commandes dans le terminal

L'arborescence des fichiers

Manipuler les fichiers

Caractères joker et motifs du shell

Développement de noms de chemins

Caractères joker et motifs du shell (1)

*

?

[...]

[!...]

- ▶ L'étoile * correspond à (en anglais *match*) n'importe quelle chaîne de caractère.
- ▶ a*

Caractères joker et motifs du shell (1)

*

?

[...]

[!...]

- ▶ L'étoile * correspond à (en anglais *match*) n'importe quelle chaîne de caractère.
- ▶ `a*` $\leftrightarrow$ toute chaîne commençant par un `a` (`a`, `abricot`, ...)
- ▶ `*b*a*`

Caractères joker et motifs du shell (1)

*

?

[...]

[!...]

- ▶ L'étoile * correspond à (en anglais *match*) n'importe quelle chaîne de caractère.
- ▶ **a*** $\leftrightarrow$ toute chaîne commençant par un **a** (a, abricot, ...)
- ▶ ***b*a*** $\leftrightarrow$ toute chaîne contenant un **a** au moins un **a** après au moins un **b** (ba, babar, débobinage, ...)
- ▶ Le point d'interrogation ? correspond à n'importe quel caractère.
- ▶ **a?c**

Caractères joker et motifs du shell (1)

*

?

[...]

[!...]

- ▶ L'étoile * correspond à (en anglais *match*) n'importe quelle chaîne de caractère.
- ▶ **a*** $\leftrightarrow$ toute chaîne commençant par un **a** (a, abricot, ...)
- ▶ ***b*a*** $\leftrightarrow$ toute chaîne contenant un **a** au moins un **a** après au moins un **b** (ba, babar, débobinage, ...)
- ▶ Le point d'interrogation ? correspond à n'importe quel caractère.
- ▶ **a?c** $\leftrightarrow$ toute chaîne de trois caractères commençant par **a** et se terminant par **c** (abc, arc, a.c, ...)

Caractères joker et motifs du shell (2)

*

?

[...]

[!...]

- ▶ Un ensemble de caractères entre crochets [...] correspond à un seul caractère présent dans les crochets.
- ▶ *. [ch]

Caractères joker et motifs du shell (2)

*

?

[...]

[!...]

- ▶ Un ensemble de caractères entre crochets [...] correspond à un seul caractère présent dans les crochets.
- ▶ *. [ch] ↔ toute chaîne se terminant par .c ou .h (stdio.h, cat.c, ...).
- ▶ [aeiouy]*

Caractères joker et motifs du shell (2)

*

?

[...]

[!...]

- ▶ Un ensemble de caractères entre crochets [...] correspond à un seul caractère présent dans les crochets.
- ▶ *. [ch] ↔ toute chaîne se terminant par .c ou .h (stdio.h, cat.c, ...).
- ▶ [aeiouy]* ↔ toute chaîne commençant par une voyelle

Caractères joker et motifs du shell (2)

*

?

[...]

[!...]

- ▶ Un ensemble de caractères entre crochets [...] correspond à un seul caractère présent dans les crochets.
- ▶ *. [ch] ↔ toute chaîne se terminant par .c ou .h (stdio.h, cat.c, ...).
- ▶ [aeiouy]* ↔ toute chaîne commençant par une voyelle
- ▶ On peut abréger les suites de caractères consécutifs avec un tiret - : [a-d] remplace [abcd].
- ▶ [0-9]*

Caractères joker et motifs du shell (2)

*

?

[...]

[!...]

- ▶ Un ensemble de caractères entre crochets [...] correspond à un seul caractère présent dans les crochets.
- ▶ *. [ch] ↔ toute chaîne se terminant par .c ou .h (stdio.h, cat.c, ...).
- ▶ [aeiouy]* ↔ toute chaîne commençant par une voyelle
- ▶ On peut abréger les suites de caractères consécutifs avec un tiret - : [a-d] remplace [abcd].
- ▶ [0-9]* ↔ toute chaîne commençant par un chiffre décimal

Caractères joker et motifs du shell (2)

*

?

[...]

[!...]

- ▶ Un ensemble de caractères entre crochets [...] correspond à un seul caractère présent dans les crochets.
- ▶ *. [ch] ↔ toute chaîne se terminant par .c ou .h (stdio.h, cat.c, ...).
- ▶ [aeiouy]* ↔ toute chaîne commençant par une voyelle
- ▶ On peut abréger les suites de caractères consécutifs avec un tiret - : [a-d] remplace [abcd].
- ▶ [0-9]* ↔ toute chaîne commençant par un chiffre décimal
- ▶ Avec le caractère ! *en première position* [!...] correspond à un seul caractère **non présent** dans les crochets.
- ▶ [!a-zA-Z] ↔ un caractère qui n'est pas une lettre alphabétique (minuscule ou majuscule)

Caractères joker et motifs du shell (3)

*

?

[...]

[!...]

Et si je veux mettre] ou ! ou - entre les crochets ?

Caractères joker et motifs du shell (3)

*

?

[...]

[!...]

Et si je veux mettre] ou ! ou - entre les crochets ?

- ▶ On doit mettre] en première position (deuxième si [!...]).
- ▶ [] [] ↔

Caractères joker et motifs du shell (3)

*

?

[...]

[!...]

Et si je veux mettre] ou ! ou - entre les crochets ?

- ▶ On doit mettre] en première position (deuxième si [!...]).
- ▶ [] [] $\leftrightarrow$ l'un des caractères [ou]
- ▶ [!] [] $\leftrightarrow$

Caractères joker et motifs du shell (3)

*

?

[...]

[!...]

Et si je veux mettre] ou ! ou - entre les crochets ?

- ▶ On doit mettre] en première position (deuxième si [!...]).
- ▶ [] [] $\leftrightarrow$ l'un des caractères [ou]
- ▶ [!] [] $\leftrightarrow$ un caractère qui n'est ni [ni]
- ▶ Un ! ailleurs qu'en première position est littéral.
- ▶ [?!.]

Caractères joker et motifs du shell (3)

*

?

[...]

[!...]

Et si je veux mettre] ou ! ou - entre les crochets ?

- ▶ On doit mettre] en première position (deuxième si [!...]).
- ▶ [] [] $\leftrightarrow$ l'un des caractères [ou]
- ▶ [!] [] $\leftrightarrow$ un caractère qui n'est ni [ni]
- ▶ Un ! ailleurs qu'en première position est littéral.
- ▶ [?!.] $\leftrightarrow$ un des caractères ?, ! ou .
- ▶ [!!!]

Caractères joker et motifs du shell (3)

*

?

[...]

[!...]

Et si je veux mettre] ou ! ou - entre les crochets ?

- ▶ On doit mettre] en première position (deuxième si [!...]).
- ▶ [] [] ↔ l'un des caractères [ou]
- ▶ [!] [] ↔ un caractère qui n'est ni [ni]
- ▶ Un ! ailleurs qu'en première position est littéral.
- ▶ [?!.] ↔ un des caractères ?, ! ou .
- ▶ [!!] ↔ un caractère qui n'est pas un point d'exclamation.
- ▶ Un - en première position est littéral (deuxième si ! ou] en première position, troisième si ...)
- ▶ [-a-z]

Caractères joker et motifs du shell (3)

*

?

[...]

[!...]

Et si je veux mettre] ou ! ou - entre les crochets ?

- ▶ On doit mettre] en première position (deuxième si [!...]).
- ▶ [] [] ↔ l'un des caractères [ou]
- ▶ [!] [] ↔ un caractère qui n'est ni [ni]
- ▶ Un ! ailleurs qu'en première position est littéral.
- ▶ [?!.] ↔ un des caractères ?, ! ou .
- ▶ [!!] ↔ un caractère qui n'est pas un point d'exclamation.
- ▶ Un - en première position est littéral (deuxième si ! ou] en première position, troisième si ...)
- ▶ [-a-z] ↔ un tiret ou une lettre minuscule.
- ▶ [!]-!]

Caractères joker et motifs du shell (3)

*

?

[...]

[!...]

Et si je veux mettre] ou ! ou - entre les crochets ?

- ▶ On doit mettre] en première position (deuxième si [!...]).
- ▶ [] [] ↔ l'un des caractères [ou]
- ▶ [!] [] ↔ un caractère qui n'est ni [ni]
- ▶ Un ! ailleurs qu'en première position est littéral.
- ▶ [?!.] ↔ un des caractères ?, ! ou .
- ▶ [!!] ↔ un caractère qui n'est pas un point d'exclamation.
- ▶ Un - en première position est littéral (deuxième si ! ou] en première position, troisième si ...)
- ▶ [-a-z] ↔ un tiret ou une lettre minuscule.
- ▶ [!]-!] ↔ un caractère qui n'est ni] ni - ni !

Documentation des commandes dans le terminal

L'arborescence des fichiers

Manipuler les fichiers

Caractères joker et motifs du shell

Développement de noms de chemins

Développement de noms de chemins

- ▶ Si un mot de la ligne de commande contient au moins un des caractères *, ? ou [, le shell essaie de trouver des chemins de fichiers correspondant à un motif.
- ▶ En cas de succès, le mot est remplacé par **tous les chemins correspondants**.
- ▶ **Attention** : en cas d'échec, le mot est laissé tel quel.
- ▶ C'est le **développement de noms de chemins** (*pathname expansion*).

Exemple : si le répertoire courant contient les fichiers `exo1.c`, `exo2.c`, `truc.c` et `exo1.h`.

▶ `echo *.c`

Exemple : si le répertoire courant contient les fichiers
exo1.c, exo2.c, truc.c et exo1.h.

▶ `echo *.c`
 exo1.c exo2.c truc.c

▶ `echo *.tex`

Exemple : si le répertoire courant contient les fichiers
exo1.c, exo2.c, truc.c et exo1.h.

- ▶ `echo *.c`
 exo1.c exo2.c truc.c
- ▶ `echo *.tex`
 *.tex
- ▶ `echo ./exo[0-9].c`

Exemple : si le répertoire courant contient les fichiers
exo1.c, exo2.c, truc.c et exo1.h.

▶ `echo *.c`
 `exo1.c exo2.c truc.c`

▶ `echo *.tex`
 `*.tex`

▶ `echo ./exo[0-9].c`
 `./exo1.c ./exo2.c`

▶ `echo ?ruc.c`

Exemple : si le répertoire courant contient les fichiers
exo1.c, exo2.c, truc.c et exo1.h.

▶ `echo *.c`
 exo1.c exo2.c truc.c

▶ `echo *.tex`
 *.tex

▶ `echo ./exo[0-9].c`
 ./exo1.c ./exo2.c

▶ `echo ?ruc.c`
 truc.c

▶ `echo *.[!ch]`

Exemple : si le répertoire courant contient les fichiers
exo1.c, exo2.c, truc.c et exo1.h.

- ▶ `echo *.c`
 exo1.c exo2.c truc.c
- ▶ `echo *.tex`
 *.tex
- ▶ `echo ./exo[0-9].c`
 ./exo1.c ./exo2.c
- ▶ `echo ?ruc.c`
 truc.c
- ▶ `echo *.[!ch]`
 *.[!ch]

Deux subtilités

Lors du développement de noms de chemins,

- ▶ tous les caractères / doivent être explicitement écrits ;
- ▶ tous les points (.) en début de chaîne ou juste après un / doivent être explicitement écrits.

Les fichiers dont le nom commence par un point sont appelés des fichiers cachés.

Deux subtilités

Lors du développement de noms de chemins,

- ▶ tous les caractères / doivent être explicitement écrits ;
- ▶ tous les points (.) en début de chaîne ou juste après un / doivent être explicitement écrits.

Les fichiers dont le nom commence par un point sont appelés des fichiers cachés.

- ▶ `echo /*` affiche tous les fichiers présents dans le répertoire racine.
- ▶ `echo /*/*` affiche tous les fichiers présents dans un sous-répertoire de la racine.
- ▶ `echo .*` affiche tous les fichiers cachés présents dans le répertoire courant.

Démonstration

Fichiers cachés

- ▶ dans chaque répertoire, `.` (lien vers lui-même) et `..` (lien vers son parent)
- ▶ fichiers de configuration dans le répertoire personnel, par exemple `.bashrc`, `.vimrc`, ...
- ▶ voire des répertoires entiers cachés, par exemple `~/.mozilla`, ...

Question

Que fait la commande suivante ?

```
rm -rf ~/*
```